Company or Institution Name

Department, Team or Reference Sector

Technical Report / Project Report

Project Reference, Client or Exam

EngTeX — Engineering LaTeX Template
User Manual

Author / Team / Candidates:
Firstname Lastname
Firstname Lastname

Reviewer / Manager / Professor:
Firstname Lastname

Version 2.0

Date: July 7, 2026

Contents

1. Structure and General Settings

The template is designed to provide a complete and modular environment for writing technical reports, theses, and documentation. Before diving into the details of the individual modules (math, circuits, etc.), this chapter covers the basic features included in the template and how to configure the main document.

1.1 Document Version Management

During the drafting of a document, it is crucial to keep track of the current version. The template allows showing a version identifier directly on the title page or in the footer. In the `main_ENG_Manual.tex` file (or equivalent), simply define the command:

```
\renewcommand{\versiondoc}{2.0}
```

The template automatically links this definition to the `\mostraversione` command (or its English alias `\viewversion`), which is called to print the version (for example, in `frontespizio.tex`). If you do not wish to display the version, just leave it empty or comment it out.

1.2 Acronym Management (acro)

For technical documents, acronyms are vital. The template uses the `acro` package. Acronyms must be defined in the preamble of the main file using the `\DeclareAcronym` command:

```
\DeclareAcronym{fpga}{
  short = FPGA,
  long = Field Programmable Gate Array
}
```

In the text, you can reference the acronym using:

- `\ac{fpga}`: Prints the full form on first use (e.g. Field Programmable Gate Array (FPGA)) and the short form on subsequent uses (e.g. FPGA).
- `\acl{fpga}`: Always forces the printing of the full form (e.g. Field Programmable Gate Array).
- `\acs{fpga}`: Always forces the printing of the short form (e.g. FPGA).

To print the list of acronyms used in the document, place the `\printacronyms` command where you want (typically after the table of contents).

1.3 Bibliography Usage (biblatex)

The bibliography is managed automatically using `biblatex` with the `biber` backend. To insert sources:

1. Add your references in BibTeX format to the `layout/bibliografia.bib` file.
2. Cite sources in your text using the `\cite{bibtex_key}` command (e.g. [1] or `\parencite{paper_esempio}`
3. Print the bibliography at the end of the document by inserting the `\printbibliography` command.

1.4 Main Module: `engtex.sty`

The base package `engtex.sty` is the core of the template. It acts as an "Hub" to load all other modules and manages the language and style settings for the entire document. This allows you to selectively enable or disable template features: if you don't need to draw circuits in a project, you can simply avoid loading the corresponding module, speeding up compile times.

How to Use (Code Example)

The package is loaded in the preamble of the `main.tex` document. It uses a key-value interface for configuration.

```
\usepackage[
  language=english, % Document language: 'italian' or 'english'
  modules={math, boxes, tikz, circuits, code, algorithms} % Modules to load
]{package/engtex}
```

Available Options

- **`language`**: Automatically sets translations for algorithms, theorems, and layout labels (`italian` or `english`).
- **`modules`**: A comma-separated list of submodules to activate.

2. Boxes and Theorems Module: `engtex-boxes.sty`

This module manages the aesthetics of all information boxes and mathematical environments (theorems, definitions, etc.).

2.1 What it does

It allows you to call structured elements semantically, automatically applying a professional layout design.

2.2 How to Use (Code Example)

The syntax is: `\nomemacro{Title}{Text}` for elements requiring a title (e.g. definitions, theorems, examples), or `\nomemacro{Text}` for notes and warnings.

```
\definition{Causality}{A system is causal if...}

\note{Ensure that the sampling frequency satisfies the Nyquist-Shannon theorem.}
```

2.3 Full List of Features

Below are all the information and mathematical environments available:

- **Definition**: `\definition{Title}{Text}` | **IT**: `\definizione`
- **Theorem**: `\theorem{Title}{Text}` | **IT**: `\teorema`
- **Corollary**: `\corollary{Title}{Text}` | **IT**: `\corollario`
- **Proposition**: `\proposition{Title}{Text}` | **IT**: `\proposizione`
- **Lemma**: `\lemmaenv{Title}{Text}`
- **Proof**: `\proofenv{Text}`
- **Note**: `\note{Text}` (No title) | **IT**: `\nota`
- **Warning**: `\warning{Text}` | **IT**: `\attenzione`
- **Example**: `\example{Title}{Text}` | **IT**: `\esempio`
- **Remember**: `\remember{Text}` | **IT**: `\ricorda`
- **Tip**: `\tip{Text}` | **IT**: `\suggerimento`
- **Exam Question**: `\examquestion{Text}` | **IT**: `\domandaesame`
- **Curiosity**: `\curiosity{Text}` | **IT**: `\curiosita`

2.4 Visual Result

Definition: Causality

A system is causal if the output at time t depends only on the values of the input at times prior to or equal to t.

Note: *This is an example of a sidebar note block.*

3. Math Module: `engtex-math.sty`

This module extends standard LaTeX capabilities by providing optimized macros for frequent math and engineering formulas, reducing code verbosity.

3.1 What it does

It simplifies typing complex operators like derivatives, integrals, transforms (Fourier, Laplace, Z), and vector operators (gradient, divergence, curl), ensuring excellent spacing and typographical formatting.

3.2 How to Use (Code Example)

```
The Laplace transform of a derivative is:
\eqn{eq:laplace}{
  \laplace{\der{f(t)}{t}} = s \laplace{f(t)} - f(0)
}

Calculating curl and divergence:
\eqns{eq:vector}{
  \curl \mathbf{E} &= -\pder{\mathbf{B}}{t} \\
  \dive \mathbf{B} &= 0
}
```

3.3 Full List of Features

Below are all the math macros available:

- **Equation Systems**: `\system{ x = 1 \\y = 2 }`
- **First Derivative**: `\der{f}{t}`
- **Second Derivative**: `\dder{f}{t}`
- **N-th Derivative**: `\nder{n}{f}{t}`
- **Partial Derivative**: `\pder{f}{x}`
- **Definite Integral**: `\intab{a}{b}{f(x)}{x}`
- **Limit Evaluation**: `\eval{x^2}{0}{1}`
- **Fourier Transform**: `\fourier{x(t)}`
- **Laplace Transform**: `\laplace{f(t)}`
- **Z Transform**: `\ztrans{x[n]}`
- **Phasor**: `\phasor{V}{\phi}` or `\fasore`
- **Real and Imaginary Part**: `\real{z}`, `\imag{z}`
- **Conjugate and Negation**: `\conj{z}`, `\notlog{A}`
- **Number Sets**: `\RR, \NN, \ZZ, \CC, \QQ`
- **Norm and Absolute Value**: `\norm{v}`, `\abs{x}`

- **Inner Product**: `\inner{u}{v}`
- **Vector Operators**: `\grad` (∇), `\dive` ($\nabla\cdot$), `\curl` ($\nabla\times$), `\lap` (∇^2)
- **Matrix Operations**: `\tr{A}`, `\det{A}`, `\rank{A}`, `\adj{A}`, `\matrixthree{1}...{9}`
- **Probability and Statistics**: `\prob{A}`, `\expval{X}`, `\var{X}`, `\cov{X}{Y}`
- **Limits, Sums, Products**: `\limit{x}{0}`, `\summ{i}{N}`, `\prodd{i}{N}`
- **Numbered Equations**: `\eqn{label}{...}` and for systems `\eqns{label}{...}`

3.4 Visual Result

The Laplace transform of a derivative is:

$$\mathcal{L}\left\{\frac{df(t)}{dt}\right\} = s\mathcal{L}\left\{f(t)\right\} - f(0) \tag{3.1}$$

Calculating curl and divergence:

$$\nabla \times \mathbf{E} = -\frac{\partial \mathbf{B}}{\partial t} \tag{3.2}$$

$$\nabla \cdot \mathbf{B} = 0 \tag{3.3}$$

4. Code Module: `engtex-code.sty`

Inserting code snippets in a technical report requires excellent readability and correct syntax highlighting. The `engtex-code` module leverages `listings` to provide pre-configured code blocks for developers and engineers.

4.1 What it does

It allows you to insert source code formatted with professional color schemes. Optimized pre-configured languages include: VHDL (Vivado style colors), C/C++, Python, Java, MATLAB, NASM, and Bash scripts. It also supports inline syntax for referencing variables or functions in plain text.

4.2 How to Use (Code Example)

```
An example of Python code (use \pythoninline{import numpy as np} in the text):

\begin{lstlisting}[style=python]
def calculate_mean(values):
    # Returns the arithmetic mean
    return sum(values) / len(values)
\end{lstlisting}
```

4.3 Full List of Features

Below are all the supported languages and their inline commands:

- **VHDL**: `\begin{lstlisting}[style=vhdl]` | Compact: `[style=vhdlcompact]` | Inline: `\vhdlinline{...}`
- **C**: `\begin{lstlisting}[style=c]` | Compact: `[style=ccompact]` | Inline: `\cinline{...}`
- **C++**: `\begin{lstlisting}[style=cpp]` | Compact: `[style=cppcompact]` | Inline: `\cppinline{...}`
- **Python**: `\begin{lstlisting}[style=python]` | Compact: `[style=pythoncompact]` | Inline: `\pythoninline{...}`
- **Java**: `\begin{lstlisting}[style=java]` | Compact: `[style=javacompact]` | Inline: `\javainline{...}`
- **MATLAB**: `\begin{lstlisting}[style=matlab]` | Compact: `[style=matlabcompact]` | Inline: `\matlabinline{...}`
- **LaTeX**: `\begin{lstlisting}[style=latex]` | Compact: `[style=latexcompact]`
- **Bash**: `\begin{lstlisting}[style=bash]` | Inline: `\bashinline{...}`

4.4 Visual Result

An example of Python code (use `import numpy as np` in the text):

```python
def calculate_mean(values):
    # Returns the arithmetic mean
    return sum(values) / len(values)
```

```vhdl
entity D_FlipFlop is
   port ( clk, d : in std_logic; q : out std_logic );
end entity;
```

Listing 4.1: *D Flip Flop declaration in VHDL*

5. Algorithms Module: `engtex-algorithms.sty`

When describing the logical architecture of software, pseudocode is the most formal and elegant way. The `engtex-algorithms` module wraps the powerful `algorithm2e` package.

5.1 What it does

It allows you to lay out structured pseudocode blocks, complete with automatic numbering. The advantage of EngTeX is instant localization: if you set `language=english` in `engtex.sty`, all keywords (*Input, Output, if, then, while*) are printed in English.

5.2 How to Use (Code Example)

```
\begin{algorithm}[H]
  \KwIn{An array $A$ of $n$ numbers}
  \KwOut{The maximum value}
  $max \gets A[0]$\;
  \For{$i \gets 1$ \KwTo $n-1$}{
    \If{$A[i] > max$}{
      $max \gets A[i]$\;
    }
  }
  \Return{$max$}\;
  \caption{Maximum Search}
\end{algorithm}
```

5.3 Full List of Features

The package automatically translates the standard commands of `algorithm2e`:

- **Input and Output**: `\KwIn{...}`, `\KwOut{...}`
- **For Loop**: `\For{condition}{...}`
- **While Loop**: `\While{condition}{...}`
- **If-Else Construct**: `\If{...}{...}`, `\ElseIf{...}{...}`, `\Else{...}`
- **Value Return**: `\Return{...}`
- **Assignment**: Use `\gets` to print the symbol $\leftarrow$

5.4 Visual Result

Algorithm 1: Maximum Search

Input: An array A of n numbers
Output: The maximum value

$max \leftarrow A[0]$;
for $i \leftarrow 1$ **to** $n-1$ **do**
  if $A[i] > max$ **then**
    $max \leftarrow A[i]$;
return max;

6. Circuits Module: `engtex-circuits.sty`

Drawing professional electrical circuits directly in LaTeX is easy thanks to the `engtex-circuits` module, built on top of the standard `circuitikz` package.

6.1 What it does

This module loads the `circuitikz` package in European style and sets the thickness and dimensions of bipoles to ensure a clean aesthetic consistent with the rest of the document.

6.2 How to Use (Code Example)

To draw a circuit, use the standard `circuitikz` environment. Remember to wrap the circuit in a `figure` environment for proper numbering and captioning.

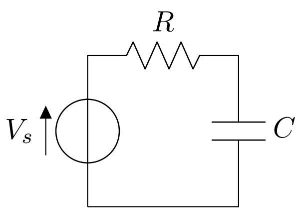
```
\begin{center}
  \begin{circuitikz}[scale=1.0]
    \draw (0,0) to[V=$V_s$] (0,2)
          to[R=$R$] (2,2)
          to[C=$C$] (2,0)
          to[short] (0,0);
  \end{circuitikz}
\end{center}
```

6.3 Pre-built Macros

The package also includes six ready-to-use macros to quickly insert common circuits by simply specifying component values. The general usage includes parameters for scaling, component values, and input/output voltages or currents.

- **Inverting Amplifier**: `\drawInvertingAmp[scale]{R_in}{R_f}{V_in}{V_out}`
- **Non-Inverting Amplifier**: `\drawNonInvertingAmp[scale]{R_1}{R_2}{V_in}{V_out}`
- **Low-Pass Filter**: `\drawLowPassFilter[scale]{R}{C}{V_in}{V_out}`
- **High-Pass Filter**: `\drawHighPassFilter[scale]{C}{R}{V_in}{V_out}`
- **Series RLC**: `\drawRLCSeries[scale]{V_s}{R}{L}{C}`
- **Parallel RLC**: `\drawRLCParallel[scale]{I_s}{R}{L}{C}`

6.4 Visual Result



7. TikZ Module: `engtex-tikz.sty`

Writing pure TikZ code to create simple block diagrams or flowcharts is tedious. The `engtex-tikz` module provides high-level semantic commands to simplify the creation of graphic nodes consistent with the document's color style.

7.1 What it does

It defines four TikZ macros that eliminate the need to write `\node[...]` code manually. The macros produce consistent graphic blocks that can be connected using standard TikZ arrows.

7.2 How to Use (Code Example)

All four macros share the same three-parameter signature: `\command{id}{text}{positioning}`

- `id`: unique identifier for the node, used to connect it to other elements (e.g. `alu1`, `dec_valid`).
- `text`: label displayed inside the block.
- `positioning`: TikZ relative coordinate (e.g. `right=of filter`, `below=1.5cm of start`). Leave this parameter empty for the first node (`{}`).

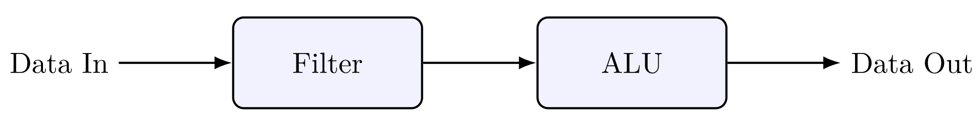
```
\begin{tikzpicture}[node distance=1.5cm, arrow/.style={-Latex, thick}]
  \node (input) {Data In};
  \sysblock{filter}{Filter}{right=of input};
  \sysblock{alu}{ALU}{right=of filter};
  \node[right=1.5cm of alu] (output) {Data Out};

  \draw[arrow] (input) -- (filter);
  \draw[arrow] (filter) -- (alu);
  \draw[arrow] (alu) -- (output);
\end{tikzpicture}
```

7.3 Full List of Features

Available commands are:

- **Start/End Block (Red rounded rectangle)**: `\startendblock{id}{text}{pos}`
- **System Block (Blue rectangle)**: `\sysblock{id}{text}{pos}`
- **Process Block (Orange rectangle)**: `\flowblock{id}{text}{pos}`
- **Decision Block (Green diamond)**: `\decblock{id}{text}{pos}`

7.4 Visual Result



8. Quick Reference and Cheat Sheet

8.1 Module Summary

Module: `math`

Code	Function	Use Case
`\system{x=1 \\y=2}`	$\begin{cases} x = 1 \\ y = 2 \end{cases}$	Equation systems
`\der{f}{t}`	$\frac{df}{dt}$	Ordinary derivatives
`\dder{f}{t}`	$\frac{d^2 f}{dt^2}$	Second derivatives
`\nder{n}{f}{t}`	$\frac{d^n f}{dt^n}$	Higher-order derivatives
`\pder{f}{x}`	$\frac{\partial f}{\partial x}$	Partial derivatives
`\intab{a}{b}{f(x)}{x}`	$\int_a^b f(x)\,dx$	Definite integrals
`\eval{x^2}{0}{1}`	$\left[x^2\right]_0^1$	Limits of integration
`\limit{x}{0}`	$\lim_{x\to 0}$	Calculus limits
`\summ{i}{N}`	$\sum_{i=1}^{N}$	Series summation
`\prodd{i}{N}`	$\prod_{i=1}^{N}$	Product operators
`\fourier{x(t)}`	$\mathcal{F}\left\{x(t)\right\}$	Signal Fourier transforms
`\laplace{f(t)}`	$\mathcal{L}\left\{f(t)\right\}$	Control Laplace transforms
`\ztrans{x[n]}`	$\mathcal{Z}\left\{x[n]\right\}$	Discrete Z transforms
`\fasore{V}{\phi}`	$V\angle\phi$	AC circuits calculations
`\real{z}, \imag{z}`	$\Re\left\{z\right\}, \Im\left\{z\right\}$	Complex analysis
`\conj{z}`	$\overline{z}$	Complex analysis
`\notlog{A \cdot B}`	$\overline{A \cdot B}$	Digital electronics
`\RR, \NN, \ZZ, \CC, \QQ`	$\mathbb{R}, \mathbb{N}, \mathbb{Z}, \mathbb{C}, \mathbb{Q}$	Common number sets
`\norm{v}, \abs{x}`	$\left\lVert v\right\rVert, \left\lvert x\right\rvert$	Algebra & calculus
`\inner{u}{v}`	$\langle u, v\rangle$	Vector inner product
`\grad f, \dive \mathbf{v}, \curl \mathbf{v}`	$\nabla f, \nabla\cdot\mathbf{v}, \nabla\times\mathbf{v}$	Vector calculus
`\lap f`	$\nabla^2 f$	Laplacian wave eq.
`\tr{A}, \det{A}`	$\mathrm{tr}(A), \det(A)$	Linear algebra
`\rank{A}, \adj{A}`	$\mathrm{rk}(A), \mathrm{adj}(A)$	Linear algebra
`\matrixthree{1}..{9}`	$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$	Matrices insertion
`\prob{A}, \expval{X}`	$P(A), E[X]$	Probability
`\var{X}, \cov{X}{Y}`	$\mathrm{Var}(X), \mathrm{Cov}(X, Y)$	Statistics
`\eqn{lbl}{..}`	Eq. (1)	Numbered equation
`\eqns{lbl}{..}`	Eq. (1), (2)	Aligned equations

Module: `boxes`

Code	Function	Use Case
`\definition` / `\definizione`	Definition Box	Core concept
`\theorem` / `\teorema`	Theorem Box	Mathematical rules
`\corollary` / `\corollario`	Corollary Box	Direct consequences
`\proposition` / `\proposizione`	Proposition Box	Minor statements
`\lemmaenv`	Lemma Box	Intermediate steps
`\proofenv`	Proof Box	Proofs
`\note` / `\nota`	Note Box	Sidebar annotations
`\warning` / `\attenzione`	Warning Box	Cautions and mistakes
`\example` / `\esempio`	Example Box	Practical examples
`\remember` / `\ricorda`	Remember Box	Important summaries
`\tip` / `\suggerimento`	Tip Box	Tricks and advice
`\examquestion` / `\domandaesame`	Exam Question Box	Study annotations
`\curiosity` / `\curiosita`	Curiosity Box	Historical trivia

Modules: `code` & `algorithms`

Code	Function	Use Case
`\begin{lstlisting} [style=python]`	Python Block	Analysis scripts
`\begin{lstlisting} [style=vhdl]`	VHDL Block	FPGA coding
`\begin{lstlisting} [style=c]`	C Block	Embedded systems
`\begin{lstlisting} [style=cpp]`	C++ Block	Object-oriented code
`\begin{lstlisting} [style=java]`	Java Block	Software design
`\begin{lstlisting} [style=matlab]`	MATLAB Block	Data & control systems
`\begin{lstlisting} [style=bash]`	Bash Block	Scripting/DevOps
`\begin{lstlisting} [style=latex]`	LaTeX Block	Technical manuals
`\begin{lstlisting} [style=`*lang*`compact]`	Compact code	Long listings (e.g. `pythoncompact`)
`\`*lang*`inline{code}`	Inline code	Variables in text
`\begin{algorithm}`	Pseudocode block	Abstract logic
`\KwIn{...}`	Input / Output	Algorithm setup
`\For{cond}{...}`	FOR Loop	Known iterations
`\While{cond}{...}`	WHILE Loop	Unknown iterations
`\If{...}{...}`	IF / ELSE	Logical conditions
`\Return{...}`	Output	Terminate function

Module: `tikz`

Code	Function	Use Case
`\startendblock{id}{text}{pos}`	Start/end block	Flowcharts (red)
`\sysblock{id}{text}{pos}`	System block	Hardware/registers (blue)
`\flowblock{id}{text}{pos}`	Process block	Software tasks (orange)
`\decblock{id}{text}{pos}`	Decision block	Conditional branches (green)

8.2 Visual Cheat Sheet: Box Styles

Below is the visual appearance of all boxes and theorems provided by the template.

Definition: Definition

Example of a definition box.

Theorem 8.1: Theorem

Example of a theorem box.

Lemma 8.1: Lemma

Example of a lemma box.

Proof:
Example of a proof box. ■

Note: *Example of a note box.*

Warning: Example of a warning box.

Example 8.1 – Example

Example of a numbered example box.

Remember

Remember this information.

Tip

A very useful tip for you.

Exam Question

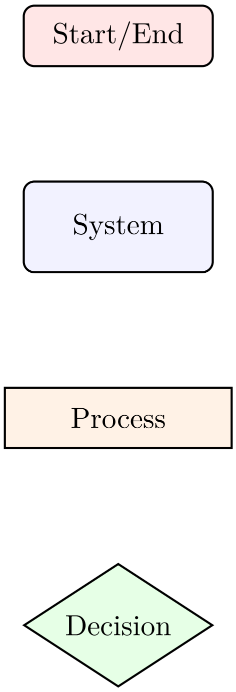
This will definitely be in the exam.

Fun Fact

Did you know that...

8.3 Visual Cheat Sheet: TikZ Blocks

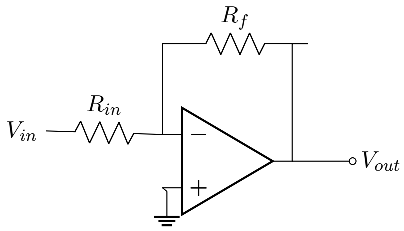
Below is the visual appearance of the semantic TikZ blocks.



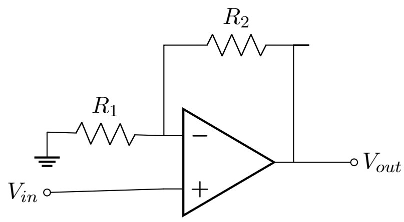
8.4 Visual Cheat Sheet: Electrical Circuits

Below are the pre-built macros for drawing circuits quickly.

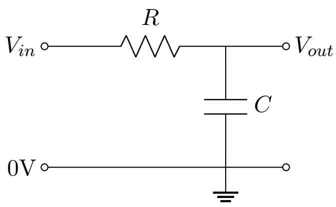
Inverting Amplifier



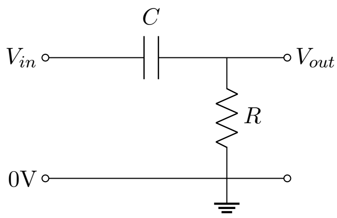
Non-Inverting Amplifier



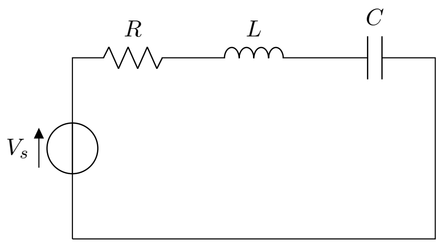
Low-Pass Filter



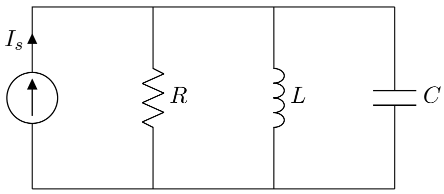
High-Pass Filter



Series RLC



Parallel RLC



References

[1] A. Turing, “Computing machinery and intelligence,” *Mind*, vol. 59, no. 236, pp. 433–460, 1950.